

Complete Sotility Smart Contract Suite

This document provides a comprehensive overview of the complete Sotility smart contract ecosystem, organizing all contracts by functional category for clarity.

Table of Contents

1. Token Contracts

- [SotilityStableToken.sol](#)
- [SotilityOwnershipToken.sol](#)
- [SoGoodUtilityGovernance.sol](#)
- [SotilityVeToken.sol](#)
- [SotilityLiquidStaking.sol](#)

2. Social Layer Contracts

- [SoGoodFeed.sol](#)
- [SotilityProfileRegistry.sol](#)
- [SotilityBadgeNFT.sol](#)
- [SotilityContentGovernance.sol](#)

3. Financial Infrastructure Contracts

- [SotilityExchange.sol](#)
- [SotilityTreasuryRouter.sol](#)
- [SotilityYieldEngine.sol](#)
- [SotilityVaultFactory.sol](#)
- [SotilityBridgeAdapter.sol](#)
- [SotilityAdaptiveTokenomics.sol](#)

4. AI and Oracle Contracts

- [AIOracleManager.sol](#)
- [MultiOracleAggregator.sol](#)

5. Identity and Reputation Contracts

- [SotilityZKIdentity.sol](#)
- [SotilityCrossChainIdentity.sol](#)
- [SotilityProofOfPersonhood.sol](#)

6. Security and Risk Management Contracts

- SotilityEmergencyShutdown.sol
- SotilityInsurance.sol

7. Deployment and Integration Strategy

Token Contracts

1. SotilityStableToken.sol

Plain Text

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityStableToken (SST)
 * @dev Pegged stable token backed 1:1 by verified revenue.
 * Minted by DAO-approved minters and tracked with IPFS receipts.
 */
contract SotilityStableToken is ERC20Burnable, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    string public receiptBaseUri;

    struct MintReceipt {
        address to;
        uint256 amount;
        string ipfsHash;
        uint256 timestamp;
    }

    MintReceipt[] public mintReceipts;

    event Minted(address indexed to, uint256 amount, string ipfsHash);
    event BaseUriUpdated(string newBase);

    constructor() ERC20("Sotility Stable Token", "SST") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function mint(address to, uint256 amount, string memory ipfsHash)
        external onlyRole(MINTER_ROLE) {
        _mint(to, amount);
        mintReceipts.push(MintReceipt({
```

```

        to: to,
        amount: amount,
        ipfsHash: ipfsHash,
        timestamp: block.timestamp
    )));
    emit Minted(to, amount, ipfsHash);
}

function setBaseUri(string calldata uri) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    receiptBaseUri = uri;
    emit BaseUriUpdated(uri);
}

function getReceipts() external view returns (MintReceipt[] memory) {
    return mintReceipts;
}

function getReceiptLink(uint256 index) external view returns (string
memory) {
    require(index < mintReceipts.length, "Invalid index");
    return string(abi.encodePacked(receiptBaseUri,
mintReceipts[index].ipfsHash));
}
}

```

2. SotilityOwnershipToken.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityOwnershipToken (SOT)
 * @dev Represents equity stake in Sotility Ecosystem. Tradable and dividend-
eligible.
 */
contract SotilityOwnershipToken is ERC20Burnable, AccessControl {
    bytes32 public constant TREASURY_ROLE = keccak256("TREASURY_ROLE");

    mapping(address => uint256) public lastClaimed;
    uint256 public dividendPerToken;
    uint256 public totalDistributed;
}

```

```

event DividendsDistributed(uint256 amount);
event DividendsClaimed(address indexed user, uint256 amount);

constructor() ERC20("Sotility Ownership Token", "SOT") {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}

function mint(address to, uint256 amount) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    _mint(to, amount);
}

function distributeDividends() external payable onlyRole(TREASURY_ROLE) {
    require(totalSupply() > 0, "No tokens in circulation");
    dividendPerToken += (msg.value * 1e18) / totalSupply();
    totalDistributed += msg.value;
    emit DividendsDistributed(msg.value);
}

function claimDividends() external {
    uint256 owed = getOwedDividends(msg.sender);
    require(owed > 0, "No dividends owed");
    lastClaimed[msg.sender] = dividendPerToken;
    payable(msg.sender).transfer(owed);
    emit DividendsClaimed(msg.sender, owed);
}

function getOwedDividends(address user) public view returns (uint256) {
    uint256 delta = dividendPerToken - lastClaimed[user];
    return (balanceOf(user) * delta) / 1e18;
}

receive() external payable {
    revert("Use distributeDividends()");
}
}

```

3. SoGoodUtilityGovernance.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**

```

```

* @title SoGoodUtilityGovernance (SUG)
* @dev Governance, Learn2Earn, Staking, and Proof-of-Utility tipping token.
*/
contract SoGoodUtilityGovernance is ERC20Burnable, AccessControl {
    bytes32 public constant TIP_ROLE = keccak256("TIP_ROLE");
    bytes32 public constant DISTRIBUTOR_ROLE = keccak256("DISTRIBUTOR_ROLE");

    mapping(address => uint256) public unlockTimestamp;
    uint256 public immutable lockDuration;

    event Tipped(address indexed from, address indexed to, uint256 amount,
string reason);
    event Airdropped(address indexed user, uint256 amount);

    constructor(uint256 _lockDuration) ERC20("SoGood Utility Governance",
"SUG") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        lockDuration = _lockDuration; // e.g. 60 days = 5184000 seconds
    }

    function airdrop(address user, uint256 amount) external
onlyRole(DISTRIBUTOR_ROLE) {
        _mint(user, amount);
        unlockTimestamp[user] = block.timestamp + lockDuration;
        emit Airdropped(user, amount);
    }

    function tip(address to, uint256 amount, string calldata reason)
external onlyRole(TIP_ROLE) {
        _mint(to, amount);
        emit Tipped(msg.sender, to, amount, reason);
    }

    function transfer(address to, uint256 amount) public override returns
(bool) {
        require(block.timestamp >= unlockTimestamp[msg.sender], "Tokens are
time-locked");
        return super.transfer(to, amount);
    }

    function transferFrom(address from, address to, uint256 amount) public
override returns (bool) {
        require(block.timestamp >= unlockTimestamp[from], "Tokens are time-
locked");
        return super.transferFrom(from, to, amount);
    }

    function isUnlocked(address user) external view returns (bool) {

```

```

        return block.timestamp >= unlockTimestamp[user];
    }
}

```

4. SotilityVeToken.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityVeToken
 * @dev Vote-escrowed token model for longer-term governance commitment
 */
contract SotilityVeToken is AccessControl {
    bytes32 public constant TOKEN_MANAGER_ROLE =
    keccak256("TOKEN_MANAGER_ROLE");

    struct LockPosition {
        uint256 amount;
        uint256 lockEnd;
        uint256 veAmount;
    }

    // Token being locked
    address public immutable baseToken;

    // Maximum lock time (4 years)
    uint256 public constant MAX_LOCK_TIME = 4 * 365 days;

    // User balances
    mapping(address => LockPosition) public lockPositions;

    // Total veToken supply
    uint256 public totalVeSupply;

    event TokensLocked(address indexed user, uint256 amount, uint256
lockEnd, uint256 veAmount);
    event LockIncreased(address indexed user, uint256 newAmount, uint256
newLockEnd, uint256 newVeAmount);
    event TokensUnlocked(address indexed user, uint256 amount);

    constructor(address _baseToken) {
        baseToken = _baseToken;
    }
}

```

```

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
        _grantRole(TOKEN_MANAGER_ROLE, msg.sender);
    }

    /**
     * @dev Lock tokens for a period of time to receive veTokens
     * @param amount Amount of base tokens to lock
     * @param lockDuration Duration to lock tokens in seconds (up to
MAX_LOCK_TIME)
     */
    function lockTokens(uint256 amount, uint256 lockDuration) external {
        require(amount > 0, "Amount must be greater than 0");
        require(lockDuration > 0 && lockDuration <= MAX_LOCK_TIME, "Invalid
lock duration");

        IERC20(baseToken).transferFrom(msg.sender, address(this), amount);

        uint256 lockEnd = block.timestamp + lockDuration;
        uint256 veAmount = calculateVeAmount(amount, lockDuration);

        LockPosition storage position = lockPositions[msg.sender];

        if (position.amount > 0) {
            // User already has a lock position, merge them
            position.amount += amount;
            position.lockEnd = lockEnd > position.lockEnd ? lockEnd :
position.lockEnd;

            // Recalculate veAmount based on new values
            uint256 remainingDuration = position.lockEnd - block.timestamp;
            position.veAmount = calculateVeAmount(position.amount,
remainingDuration);
        } else {
            // New lock position
            position.amount = amount;
            position.lockEnd = lockEnd;
            position.veAmount = veAmount;
        }

        totalVeSupply += veAmount;

        emit TokensLocked(msg.sender, amount, lockEnd, veAmount);
    }

    /**
     * @dev Extend lock duration to get more veTokens
     * @param newDuration New lock duration in seconds from now
     */

```

```

function extendLock(uint256 newDuration) external {
    LockPosition storage position = lockPositions[msg.sender];
    require(position.amount > 0, "No existing lock");
    require(position.lockEnd > block.timestamp, "Lock expired");
    require(newDuration > 0, "Duration must be greater than 0");

    uint256 newLockEnd = block.timestamp + newDuration;
    require(newLockEnd > position.lockEnd, "New lock end must be later
than current");
    require(newLockEnd <= block.timestamp + MAX_LOCK_TIME, "Lock too
long");

    // Remove old veTokens from total supply
    totalVeSupply -= position.veAmount;

    // Update lock position
    position.lockEnd = newLockEnd;
    position.veAmount = calculateVeAmount(position.amount, newDuration);

    // Add new veTokens to total supply
    totalVeSupply += position.veAmount;

    emit LockIncreased(msg.sender, position.amount, newLockEnd,
position.veAmount);
}

/**
 * @dev Unlock tokens after lock period ends
 */
function unlock() external {
    LockPosition storage position = lockPositions[msg.sender];
    require(position.amount > 0, "No existing lock");
    require(block.timestamp >= position.lockEnd, "Lock not expired yet");

    uint256 amount = position.amount;

    // Remove veTokens from total supply
    totalVeSupply -= position.veAmount;

    // Clear lock position
    position.amount = 0;
    position.lockEnd = 0;
    position.veAmount = 0;

    // Return tokens to user
    IERC20(baseToken).transfer(msg.sender, amount);

    emit TokensUnlocked(msg.sender, amount);
}

```

```

}

/**
 * @dev Calculate veToken amount based on lock duration
 * @param amount Base token amount
 * @param duration Lock duration in seconds
 */
function calculateVeAmount(uint256 amount, uint256 duration) public pure
returns (uint256) {
    // Linear scaling: 1 year = 0.25x, 2 years = 0.5x, 3 years = 0.75x,
4 years = 1x
    return (amount * duration) / MAX_LOCK_TIME;
}

/**
 * @dev Get current veToken balance for an account
 * @param account Address to check
 */
function balanceOf(address account) external view returns (uint256) {
    LockPosition storage position = lockPositions[account];

    if (position.lockEnd <= block.timestamp) {
        return 0; // Lock expired, no voting power
    }

    return position.veAmount;
}

/**
 * @dev Get detailed lock position info
 */
function getLockPosition(address account) external view returns (
    uint256 amount,
    uint256 lockEnd,
    uint256 veAmount,
    uint256 remainingTime
) {
    LockPosition storage position = lockPositions[account];
    uint256 remaining = position.lockEnd > block.timestamp ?
position.lockEnd - block.timestamp : 0;

    return (
        position.amount,
        position.lockEnd,
        position.veAmount,
        remaining
    );
}

```

```
}  
}
```

5. SotilityLiquidStaking.sol

Plain Text

```
// SPDX-License-Identifier: MIT  
pragma solidity ^0.8.20;  
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";  
import "@openzeppelin/contracts/access/AccessControl.sol";  
  
/**  
 * @title SotilityLiquidStaking  
 * @dev Allows users to stake SOT while receiving liquid staked tokens  
 (lstSOT)  
 */  
contract SotilityLiquidStaking is ERC20, AccessControl {  
    bytes32 public constant VALIDATOR_ROLE = keccak256("VALIDATOR_ROLE");  
    bytes32 public constant FEE_COLLECTOR_ROLE =  
keccak256("FEE_COLLECTOR_ROLE");  
  
    address public immutable stakingToken; // SOT  
  
    uint256 public protocolFee; // Basis points (100 = 1%)  
    uint256 public totalStaked;  
    uint256 public totalRewards;  
    uint256 public constant FEE_DENOMINATOR = 10000;  
  
    event Staked(address indexed user, uint256 amount, uint256 lstAmount);  
    event Unstaked(address indexed user, uint256 lstAmount, uint256  
sotAmount);  
    event RewardsAdded(uint256 amount);  
    event ProtocolFeeUpdated(uint256 newFee);  
  
    constructor(address _stakingToken, uint256 _initialFee) ERC20("Liquid  
Staked SOT", "lstSOT") {  
        stakingToken = _stakingToken;  
        protocolFee = _initialFee;  
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);  
        _grantRole(VALIDATOR_ROLE, msg.sender);  
        _grantRole(FEE_COLLECTOR_ROLE, msg.sender);  
    }  
  
    /**  
     * @dev Stake SOT tokens and receive lstSOT in return  
     * @param amount Amount of SOT to stake
```

```

*/
function stake(uint256 amount) external {
    require(amount > 0, "Cannot stake 0");

    // Transfer SOT from user
    IERC20(stakingToken).transferFrom(msg.sender, address(this), amount);

    // Calculate lstSOT to mint based on current exchange rate
    uint256 lstAmount = (amount * 1e18) / exchangeRate();

    // Mint lstSOT to user
    _mint(msg.sender, lstAmount);

    // Update total staked
    totalStaked += amount;

    emit Staked(msg.sender, amount, lstAmount);
}

/**
 * @dev Unstake SOT by burning lstSOT
 * @param lstAmount Amount of lstSOT to burn
 */
function unstake(uint256 lstAmount) external {
    require(lstAmount > 0, "Cannot unstake 0");
    require(balanceOf(msg.sender) >= lstAmount, "Insufficient lstSOT
balance");

    // Calculate SOT amount based on current exchange rate
    uint256 sotAmount = (lstAmount * exchangeRate()) / 1e18;

    // Burn lstSOT
    _burn(msg.sender, lstAmount);

    // Calculate protocol fee
    uint256 feeAmount = (sotAmount * protocolFee) / FEE_DENOMINATOR;
    uint256 sotToReturn = sotAmount - feeAmount;

    // Transfer SOT back to user
    IERC20(stakingToken).transfer(msg.sender, sotToReturn);

    // Transfer fee to fee collector
    if (feeAmount > 0) {
        address feeCollector = getRoleMember(FEE_COLLECTOR_ROLE, 0);
        IERC20(stakingToken).transfer(feeCollector, feeAmount);
    }

    // Update total staked

```

```

        totalStaked -= sotAmount;

        emit Unstaked(msg.sender, lstAmount, sotToReturn);
    }

    /**
     * @dev Add staking rewards
     * @param amount Amount of SOT to add as rewards
     */
    function addRewards(uint256 amount) external {
        require(amount > 0, "Amount must be greater than 0");

        // Transfer SOT from sender
        IERC20(stakingToken).transferFrom(msg.sender, address(this), amount);

        // Update total rewards
        totalRewards += amount;

        emit RewardsAdded(amount);
    }

    /**
     * @dev Update protocol fee
     * @param newFee New fee in basis points
     */
    function updateProtocolFee(uint256 newFee) external
    onlyRole(DEFAULT_ADMIN_ROLE) {
        require(newFee <= 1000, "Fee too high"); // Max 10%
        protocolFee = newFee;
        emit ProtocolFeeUpdated(newFee);
    }

    /**
     * @dev Calculate current exchange rate (SOT per lstSOT)
     * @return Exchange rate with 18 decimals
     */
    function exchangeRate() public view returns (uint256) {
        uint256 totalSupply = totalSupply();
        if (totalSupply == 0) {
            return 1e18; // Initial exchange rate: 1:1
        }

        // Exchange rate increases as rewards accumulate
        return ((totalStaked + totalRewards) * 1e18) / totalSupply;
    }

    /**
     * @dev Get SOT equivalent for a given lstSOT amount

```

```

    * @param lstAmount Amount of lstSOT
    * @return Equivalent SOT amount
    */
    function sotEquivalent(uint256 lstAmount) external view returns
(uint256) {
        return (lstAmount * exchangeRate()) / 1e18;
    }
}

```

Social Layer Contracts

1. SoGoodFeed.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import "@openzeppelin/contracts/utils/Counters.sol";

/**
 * @title SoGoodFeed
 * @dev AI-verified social tipping feed for the SoGood Utility Governance
(SUG) token.
 */
contract SoGoodFeed is AccessControl {
    using Counters for Counters.Counter;
    bytes32 public constant VERIFIER_ROLE = keccak256("VERIFIER_ROLE");
    Counters.Counter private _postIds;

    struct Post {
        uint256 id;
        address author;
        string contentHash;
        string metadata; // optional: type, category, or comment
        uint256 timestamp;
        uint256 totalTips;
    }

    mapping(uint256 => Post) public posts;
    mapping(uint256 => mapping(address => uint256)) public tipsByUser;

    event PostSubmitted(uint256 indexed id, address indexed author, string
contentHash, string metadata);
    event Tipped(uint256 indexed postId, address indexed tipper, uint256
amount);

```

```

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function submitPost(string calldata contentHash, string calldata
metadata) external {
        _postIds.increment();
        uint256 postId = _postIds.current();

        posts[postId] = Post({
            id: postId,
            author: msg.sender,
            contentHash: contentHash,
            metadata: metadata,
            timestamp: block.timestamp,
            totalTips: 0
        });

        emit PostSubmitted(postId, msg.sender, contentHash, metadata);
    }

    function recordTip(uint256 postId, address tipper, uint256 amount)
external onlyRole(VERIFIER_ROLE) {
        require(posts[postId].author != address(0), "Invalid post");

        tipsByUser[postId][tipper] += amount;
        posts[postId].totalTips += amount;

        emit Tipped(postId, tipper, amount);
    }

    function getPost(uint256 postId) external view returns (Post memory) {
        return posts[postId];
    }

    function totalPosts() external view returns (uint256) {
        return _postIds.current();
    }
}

```

2. SotilityProfileRegistry.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

```

```

import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityProfileRegistry
 * @dev Stores Sotilitarian Status and Utilitier Level ratings for SoGood
ecosystem participants.
 */
contract SotilityProfileRegistry is AccessControl {
    bytes32 public constant EVALUATOR_ROLE = keccak256("EVALUATOR_ROLE");

    struct Profile {
        uint256 sotilitarianStatus; // long-term contribution score
        uint256 utilitierScore; // short-term activity score
        uint8 utilitierLevel; // 0 = Bronze, 1 = Silver, 2 = Gold, 3 =
Platinum
        uint256 lastReset; // last Utilitier Level reset
    }

    mapping(address => Profile) public profiles;

    event SotilitarianProfileUpdated(address indexed user, uint256
statusScore, uint256 utilityScore, uint8 level);
    event UtilitierLevelReset(address indexed user, uint256 newScore, uint8
newLevel);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function setProfile(address user, uint256 statusScore, uint256
utilityScore, uint8 level) external onlyRole(EVALUATOR_ROLE) {
        profiles[user].sotilitarianStatus = statusScore;
        profiles[user].utilitierScore = utilityScore;
        profiles[user].utilitierLevel = level;

        emit SotilitarianProfileUpdated(user, statusScore, utilityScore,
level);
    }

    function resetUtilitierLevel(address user, uint256 newScore, uint8
newLevel) external onlyRole(EVALUATOR_ROLE) {
        profiles[user].utilitierScore = newScore;
        profiles[user].utilitierLevel = newLevel;
        profiles[user].lastReset = block.timestamp;

        emit UtilitierLevelReset(user, newScore, newLevel);
    }
}

```

```

    function getProfile(address user) external view returns (Profile memory)
    {
        return profiles[user];
    }

    function isEligibleForSotilitarianBoost(address user) external view
returns (bool) {
    return block.timestamp >= profiles[user].lastReset + 365 days;
}
}

```

3. SotilityBadgeNFT.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";
import
"@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.sol";

/**
 * @title SotilityBadgeNFT
 * @dev Soulbound NFT badges for status, impact, and tier recognition.
 */
contract SotilityBadgeNFT is ERC721URIStorage, AccessControl {
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    uint256 private _tokenIds;
    mapping(uint256 => bool) public soulbound;

    event BadgeMinted(address indexed to, uint256 tokenId, string uri);
    event BadgeRevoked(uint256 tokenId);

    constructor() ERC721("Sotility Badge NFT", "SBNFT") {
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function mint(address to, string calldata uri, bool isSoulbound)
external onlyRole(MINTER_ROLE) returns (uint256) {
        _tokenIds++;
        uint256 newId = _tokenIds;

        _mint(to, newId);
        _setTokenURI(newId, uri);
        soulbound[newId] = isSoulbound;

        emit BadgeMinted(to, newId, uri);
    }
}

```

```

        return newId;
    }

    function revoke(uint256 tokenId) external onlyRole(MINTER_ROLE) {
        _burn(tokenId);
        emit BadgeRevoked(tokenId);
    }

    // Override to disable soulbound transfer
    function _beforeTokenTransfer(
        address from,
        address to,
        uint256 tokenId,
        uint256 batchSize
    ) internal override {
        super._beforeTokenTransfer(from, to, tokenId, batchSize);
        require(from == address(0) || !soulbound[tokenId], "Soulbound: Non-transferable");
    }

    function supportsInterface(bytes4 interfaceId)
        public view override(ERC721, AccessControl)
        returns (bool)
    {
        return super.supportsInterface(interfaceId);
    }
}

```

4. SotilityContentGovernance.sol

Plain Text

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityContentGovernance
 * @dev Implements decentralized content moderation with economic incentives
 */
contract SotilityContentGovernance is AccessControl {
    bytes32 public constant MODERATOR_ROLE = keccak256("MODERATOR_ROLE");

    enum ContentStatus { Active, UnderReview, Removed, Restored }
    enum ModeratorAction { Flag, Approve, Remove, Restore }

    struct Content {

```

```
    uint256 id;
    address author;
    string contentHash;
    ContentStatus status;
    uint256 flagCount;
    uint256 approvalCount;
    uint256 lastDecisionTime;
    address[] moderatorsVoted;
}

struct ModeratorStats {
    uint256 totalVotes;
    uint256 alignedVotes
```